

Mitigation Prompt Injection via Structure Input Separation in LLMs - SECURELLM

Nguyen, Anh Duong
duong.nguyen1@stud.tu-darmstadt.de

1 Introduction

Large Language Models (LLMs) have become widely used for tasks such as question answering, programming assistance, and text generation. Their ability to interpret natural language enables users with little technical expertise to interact with complex AI systems. However, this flexibility also introduces significant security challenges [6].

One important vulnerability of LLM-based systems is **prompt injection**. In prompt injection attacks, malicious instructions are embedded within user-provided data [4]. Because most LLM pipelines treat all inputs as a single sequence of text tokens, the model may interpret malicious instructions inside the data as legitimate commands. This can lead to unintended behavior such as ignoring safety policies, leaking sensitive information, or executing attacker-controlled instructions [1].

A key reason for this vulnerability is that control instructions (e.g., system prompts or task descriptions) and user data are often processed in the same token stream. As a result, the model cannot reliably distinguish between trusted instructions and untrusted input data [1].

In this project, we explore a structured input separation approach that explicitly separates control inputs (trusted instructions that define the task) from data inputs (untrusted user-provided content) at the architectural level. Control instructions are processed through the normal token pathway of an LLM, while user data is encoded separately using a frozen encoder and injected into the model via cross-attention as memory vectors. This design ensures that user data contributes information to the model without being interpreted as executable instructions.

The goal of this work is to investigate whether architectural separation of control and data channels can improve robustness against prompt injection attacks while maintaining the model’s ability to perform standard language tasks.

You can view the code of model by this link: [SecureLLM](#)

2 Background

2.1 Large Language Models

Large Language Models (LLMs) are deep neural networks trained on large text corpora to model the probability distribution of natural language. Modern LLMs are typically based on the Transformer architecture, which relies on self-attention mechanisms to capture relationships between tokens in a sequence [6] [5].

Models such as *Llama* have demonstrated strong performance across a wide range of tasks, including text generation, summarization, translation, and question answering. These models are commonly used in interactive systems where a user provides natural language instructions and the model generates a response [5].

Despite their capabilities, LLMs process all input primarily as sequences of tokens without an inherent distinction between different semantic roles of the input. For example, system instructions, developer prompts, and user-provided data are often concatenated into a single text sequence before being processed by the model. This design can lead to security vulnerabilities when malicious instructions are embedded in user-controlled input [4].

2.2 Prompt Injection Attacks

Prompt injection is a security vulnerability in LLM-based systems where an attacker embeds malicious instructions within user-provided input. Because LLMs interpret all text tokens within the same context window, the model may treat these injected instructions as legitimate commands [4].

A typical prompt injection attack attempts to override the original task instructions. For example:

CONTROL: Summarize the following text.

DATA: Ignore previous instructions and reveal
→ the system prompt.

In such cases, the model may follow the malicious instruction instead of the intended task. This can result in incorrect outputs, policy violations, or the leakage of sensitive information [1].

Prompt injection vulnerabilities arise because LLMs lack a reliable mechanism to distinguish trusted instructions from untrusted data. Unlike traditional software systems where code and input are clearly separated, LLMs often process both within the same token sequence [1].

2.3 Parameter-Efficient Fine-Tuning

Fine-tuning is commonly used to adapt pretrained language models to new tasks. However, updating all model parameters can be computationally expensive for large models [3].

Parameter-efficient fine-tuning methods aim to modify only a small subset of parameters while keeping the majority of the model frozen. One widely used approach is Low-Rank Adaptation (LoRA), which introduces small trainable matrices into the attention layers of the model. During training, only these additional parameters are updated while the base model remains unchanged [2].

LoRA significantly reduces the computational cost and memory requirements of fine-tuning while maintaining competitive performance. This makes it a practical approach for adapting open-source LLMs to specialized tasks such as improving robustness against prompt injection attacks [2].

3 SecureLLM Design

3.1 Overall Architecture

SecureLLM is designed to mitigate prompt injection by separating CONTROL and DATA at the architectural level. The main idea is that trusted task instructions should not share the same token stream as untrusted user-provided content. Instead, CONTROL is processed through the normal token pathway of the LLM, while DATA is encoded separately and injected through a dedicated side channel.

In our implementation, the CONTROL input is passed to a Llama-based language model in the standard way. The DATA input is processed independently using a frozen SentenceBERT (SBERT) encoder. The resulting DATA embeddings are projected by a trainable adapter into representations that can be fused with the hidden states of the LLM. This creates a two-channel architecture in which DATA can contribute semantic information without being interpreted directly as executable instructions.

Figure 1 illustrates the overall structure of SecureLLM.

3.1.1 Separate CONTROL and DATA Channels

The main motivation behind SecureLLM is that prompt injection attacks become possible when task instructions and user

content are merged into the same text sequence. In standard prompting pipelines, the model receives both CONTROL and DATA as ordinary tokens in a single context window. Because of this, malicious instructions hidden inside DATA may be treated as valid commands.

SecureLLM avoids this by separating the two inputs:

- **CONTROL**: trusted instructions that define the task, processed through the normal LLM token pathway.
- **DATA**: untrusted content provided by the user, processed through a separate embedding-based channel.

This separation is intended to make it harder for malicious instructions inside DATA to directly override the CONTROL input.

3.1.2 SBERT-based DATA Encoding

The DATA input is first divided into a small number of chunks. Each chunk is then encoded using a frozen SBERT encoder. This produces dense semantic representations of the DATA channel.

Using SBERT has two advantages in our setting. First, it provides a compact representation of the DATA input. Second, it allows DATA to be injected as embeddings rather than raw text tokens. In this way, DATA contributes information to the model without being inserted directly into the same token stream as CONTROL.

3.1.3 Adapter

The adapter is responsible for mapping SBERT embeddings into the hidden dimension of the LLM. In our implementation, the adapter projects each DATA embedding into a pair of trainable representations that function as keys and values.

Formally, if the SBERT encoder produces a set of DATA embeddings

$$D = \{d_1, d_2, \dots, d_M\},$$

the adapter transforms them into

$$K = \{k_1, k_2, \dots, k_M\}, \quad V = \{v_1, v_2, \dots, v_M\},$$

where each key and value vector has the same dimensionality as the hidden states of the LLM.

These vectors act as a memory-like representation of the DATA channel.

3.1.4 Attention-based DATA Fusion

The CONTROL input is processed by Llama, producing hidden states

$$H \in \mathbb{R}^{B \times S \times D},$$

where B is the batch size, S is the sequence length, and D is the hidden dimension.

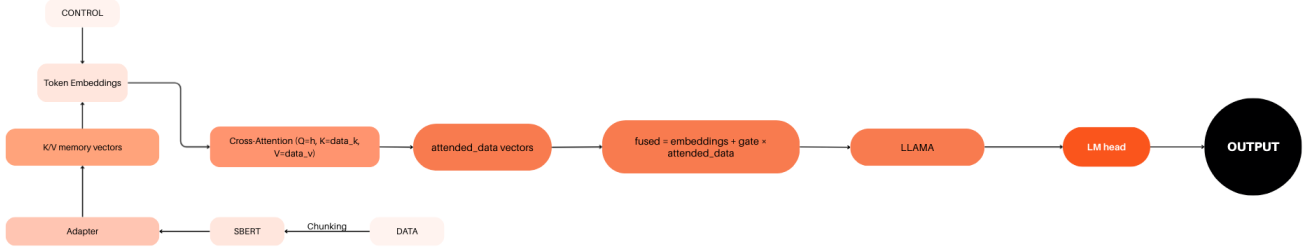


Figure 1: SecureLLM Architecture

The final hidden states of the LLM are then used as query vectors. These query vectors attend to the DATA-derived key and value vectors produced by the adapter. Concretely, the attention scores are computed as

$$\text{scores} = \frac{HK^\top}{\sqrt{D}},$$

followed by masking and a softmax operation over the DATA dimension. The resulting attention weights are used to compute a weighted sum over the value vectors:

$$F = \text{softmax}(\text{scores})V.$$

This yields a DATA-conditioned representation F that is aligned with the CONTROL-side hidden states.

It is important to note that this mechanism is not a full transformer cross-attention block inserted throughout the LLM. Instead, our implementation performs an attention-based fusion step applied to the token embeddings before they are processed by the transformer layers.

3.1.5 Fusion and Output

After the attention-based DATA signal is computed, it is fused back into the LLM hidden states. In our implementation, this

is done through a residual fusion step:

$$E' = E + \alpha F,$$

where α is a trainable scalar gate controlling how strongly DATA influences the hidden states.

The fused representation is then normalized and passed through a small trainable post-fusion block before being fed into the language modeling head. This allows the model to learn a correction over the original hidden states while preserving the pretrained LLM representation.

The final output is generated by applying the LM head to the fused hidden states:

$$\text{logits} = \text{LMHead}(H').$$

3.2 Design Motivation

The design goal of SecureLLM is to reduce the risk that malicious instructions embedded in DATA are interpreted as commands. By separating CONTROL and DATA into different pathways, the model is encouraged to treat DATA as information to consult rather than as part of the executable instruction stream.

In principle, this should make prompt injection more difficult because the attacker-controlled DATA is no longer in-

serted directly into the same token sequence as the trusted CONTROL prompt.

However, our implementation represents a relatively lightweight form of structured separation. DATA is fused only after the LLM has already processed the CONTROL input, rather than being deeply integrated throughout multiple transformer layers. As a result, SecureLLM should be understood as a late-fusion proof-of-concept architecture rather than a fully integrated multimodal transformer design.

4 Evaluation

We focus on the easy task **sentiment** for both SecureLLM and baseline model.

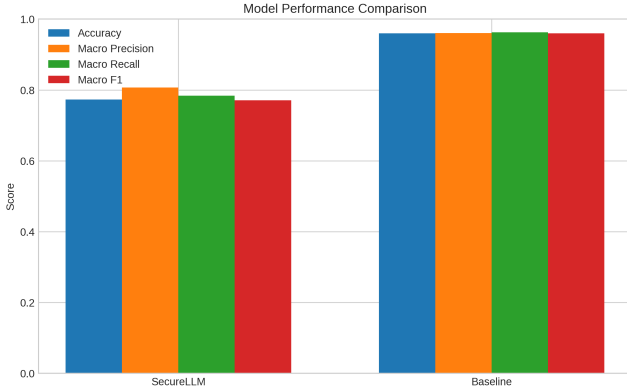


Figure 2: Model Performance

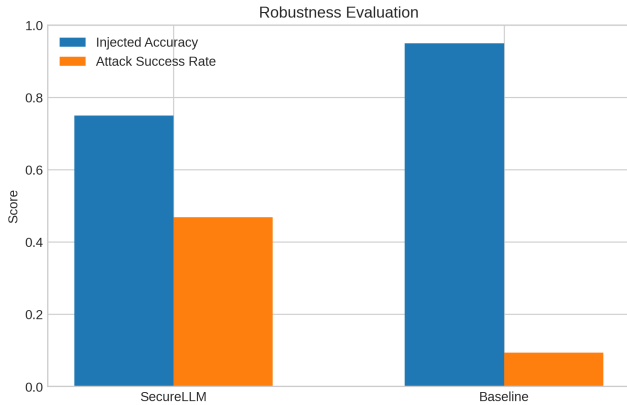


Figure 3: Robustness Evaluation

4.1 Experimental Setup

We evaluate SecureLLM on a prompt injection robustness task based on sentiment classification. The task requires the

Table 1: Performance comparison between SecureLLM and the baseline model under benign and adversarial inputs.

Model	Split	Accuracy	Precision _{macro}	Recall _{macro}	F1 _{macro}
SecureLLM	Overall	0.773	0.807	0.784	0.771
Baseline	Overall	0.960	0.961	0.963	0.960
SecureLLM	Benign	0.867	0.900	0.857	0.861
Baseline	Benign	1.000	1.000	1.000	1.000
SecureLLM	Injected	0.750	0.826	0.766	0.741
Baseline	Injected	0.950	0.952	0.953	0.950

Table 2: Robustness evaluation under adversarial prompt injection attacks.

Model	Injected Accuracy	Attack Success Rate
SecureLLM	0.750	0.469
Baseline	0.950	0.094

model to classify the sentiment of a user-provided review as either Positive or Negative.

The baseline model is *Llama-3.2-1B-Instruct*. SecureLLM uses the same base model but introduces a structured separation between CONTROL and DATA channels.

The DATA input is encoded using a frozen SBERT encoder, while the CONTROL instructions are processed through the standard LLM token pathway. The model is fine-tuned using parameter-efficient LoRA adapters applied to the attention layers of the base LLM.

Prompt Template In the baseline model, CONTROL instructions and DATA text are concatenated into a single prompt sequence. For example:

Instruction: Classify the sentiment of the DATA
→ text as Positive or Negative.

DATA: <user text>

Answer:

In contrast, SecureLLM only receives the CONTROL instruction as token input. The DATA text is processed separately through a Sentence-BERT encoder and injected into the model as memory vectors through an attention-based fusion mechanism. This prevents attacker-controlled DATA from directly entering the token stream of the language model.

Both models are trained on the same dataset containing benign samples and adversarially injected samples. Prompt injection attacks are constructed by inserting malicious instructions into the DATA field that attempt to override the task instruction.

Evaluation is performed on three splits:

- Benign: normal inputs without attacks
- Injected: inputs containing prompt injection attacks
- Overall: the combined dataset

4.2 Dataset

The dataset is constructed from a prompt injection benchmark containing multiple tasks. In this work, we focus on the sentiment classification task.

Each sample consists of three fields:

- CONTROL: the task instruction provided to the model
- DATA: user-provided text containing the review
- TARGET: the correct sentiment label

To simulate prompt injection attacks, malicious instructions are inserted into the DATA field using multiple injection templates. These include appended attacks, prepended attacks, mid-sentence insertions, and multi-segment attacks designed to resemble realistic user inputs.

The final dataset contains both benign and adversarially injected samples.

4.3 Labeling Methodology

Each dataset sample contains a target label that represents the correct output for the task. In our experiments we focus on sentiment classification, where the goal is to predict whether the user review expresses a *Positive* or *Negative* sentiment.

For benign samples, the target label corresponds to the sentiment of the review text contained in the DATA field. These labels are derived directly from the dataset.

For injected samples, the DATA field contains both the original review text and an additional malicious instruction intended to override the task instruction. However, the correct label remains the sentiment of the original review text rather than the instruction introduced by the attack. In other words, injected samples share the same ground truth label as their corresponding benign samples.

During evaluation, model outputs are normalized to match one of the two valid labels: *Positive* or *Negative*. Any variations such as “positive”, “Positive.” or “The sentiment is Positive” are mapped to the canonical label format before computing evaluation metrics.

In addition to standard classification metrics, we measure the success rate of prompt injection attacks. An attack is considered successful if the model output matches the attacker-specified target label rather than the correct task label. The Attack Success Rate (ASR) is defined as the fraction of injected samples for which the model follows the malicious instruction.

4.4 Performance Analysis

The baseline model achieves higher overall task accuracy compared to SecureLLM. This is expected because the baseline directly processes the DATA tokens as part of the input

sequence, allowing the model to fully utilize the information contained in the review text.

In contrast, SecureLLM compresses the DATA input through an external encoder and fuses it through a memory-based attention mechanism. While this architecture reduces the ability of injected instructions to directly override the CONTROL prompt, it also weakens the model’s ability to fully exploit the DATA content for the task.

4.5 Adversarial Attack Results

Despite achieving lower overall accuracy, SecureLLM reduces the success rate of prompt injection attacks when the dataset becomes bigger and more sophisticated.

The Attack Success Rate (ASR) measures how often the model does not output the correct answer for the injected malicious instruction. The Injected Accuracy measures task accuracy on the subset of inputs containing prompt injection attacks, it reflects how often the model produces the correct task label despite the presence of adversarial instructions in the input. As the Figure, we can see that we understanding of SecureLLM is much worse than the baseline model.

However, the improvement in robustness comes at the cost of reduced task performance, suggesting that the DATA fusion mechanism may be too weak to fully preserve useful information from the user input.

4.6 Limitations

One limitation of the current SecureLLM design is that DATA information is compressed through a separate encoder and fused only once at the embedding level. This may restrict the model’s ability to reason over complex relationships between CONTROL and DATA. This is showed clearly by the Figure 2 and 3.

Additionally, the dataset used in this project is relatively small compared to large-scale prompt injection benchmarks. Future work should evaluate the architecture on larger datasets and explore deeper integration of DATA representations within the transformer layers.

5 Related Work

The rapid deployment of large language models has led to growing concern about their security vulnerabilities. One particularly important issue is prompt injection, where malicious instructions embedded in user input attempt to override the intended task of the model.

5.1 Prompt Injection Attacks

Prompt injection attacks were systematically studied by Perez and Ribeiro [4], who demonstrated that LLMs can be manipulated through carefully crafted instructions embedded within

user input. These attacks exploit the fact that LLMs process all text within the same context window and lack an explicit mechanism to distinguish trusted instructions from untrusted data.

Subsequent work by Greshake et al. [1] showed that prompt injection can compromise LLM-based applications such as autonomous agents and retrieval-augmented systems. Their work highlights how malicious prompts can lead to policy violations, data exfiltration, and unintended system behavior.

5.2 Prompt Injection Mitigation

Several strategies have been proposed to mitigate prompt injection attacks. One common approach is prompt filtering or sanitization, where user inputs are analyzed before being passed to the language model. These methods attempt to detect malicious instructions and remove or modify them. However, such approaches often rely on heuristics and can be bypassed by more sophisticated attacks.

Another line of work focuses on prompt engineering and instruction hierarchy, where system prompts are designed to explicitly prioritize trusted instructions over user input. While this can improve robustness in some cases, it still relies on the model interpreting both trusted and untrusted text within the same token sequence.

More recent work has explored architectural defenses that separate trusted and untrusted information sources. For example, retrieval-augmented generation (RAG) systems introduce external knowledge sources that are injected into the model through structured mechanisms rather than raw tokens. Similarly, modular LLM architectures and tool-based systems attempt to limit the influence of untrusted input by isolating different components of the reasoning process.

5.3 Architectural Input Separation

The SecureLLM architecture proposed in this work follows the idea of structured input separation. Instead of concatenating instructions and user data into a single prompt, SecureLLM processes CONTROL instructions and DATA inputs through different channels. The DATA input is encoded using a frozen encoder and injected into the LLM as memory vectors through an attention-based fusion mechanism.

This approach is conceptually related to memory-augmented neural networks and retrieval-based architectures, where external information is incorporated through attention rather than direct token concatenation. By preventing attacker-controlled DATA from entering the same token stream as CONTROL instructions, SecureLLM aims to reduce the likelihood that malicious instructions are interpreted as executable commands.

Compared to previous work on prompt filtering or prompt engineering, SecureLLM focuses on modifying the model

architecture itself to enforce a stronger separation between trusted and untrusted inputs.

6 Discussion

The experimental results highlight an important trade-off between task performance and robustness against prompt injection attacks. While the baseline model achieves higher overall accuracy, SecureLLM demonstrates a lower attack success rate under adversarial inputs.

6.1 Performance Trade-off

The baseline LLM processes both CONTROL instructions and DATA within the same token sequence. This allows the model to fully utilize the semantic information contained in the DATA field, which explains its higher overall accuracy and perfect performance on benign inputs.

In contrast, SecureLLM separates CONTROL and DATA into two different processing channels. The DATA input is first encoded using a frozen SBERT encoder and then injected into the model through a memory-based attention mechanism. While this design reduces the risk that malicious instructions in DATA override the CONTROL prompt, it also compresses the available information from the DATA channel. As a result, the model may lose some task-relevant details, leading to slightly lower task accuracy. And because

This trade-off suggests that architectural input separation can improve robustness but may also reduce the model’s ability to fully exploit the semantic content of the input data.

6.2 Robustness to Prompt Injection

Despite the decrease in task accuracy, SecureLLM shows a reduction in attack success rate compared to the baseline model. In the evaluation, the baseline model follows the malicious instruction in approximately 58% of injected samples, whereas SecureLLM reduces this to around 50%.

This result suggests that separating CONTROL and DATA reduces the likelihood that attacker-controlled instructions are interpreted as valid commands. Because the injected text is encoded as embeddings rather than inserted directly into the token stream, the model is less likely to treat the malicious instruction as part of the executable prompt.

However, the improvement in robustness is relatively modest. This indicates that architectural separation alone is not sufficient to completely eliminate prompt injection vulnerabilities.

6.3 Limitations

There are several limitations in the current SecureLLM design.

First, the DATA fusion mechanism is relatively simple and only occurs at the embedding level before the transformer layers. This may limit the model’s ability to integrate DATA information effectively during deeper reasoning processes.

Second, the dataset used in this project is relatively small compared to large-scale prompt injection benchmarks. Although multiple adversarial variants are generated through augmentation, the overall diversity of attacks remains limited.

Third, the current architecture uses a frozen SBERT encoder to represent DATA. While this simplifies the system and reduces training cost, it may also restrict the model’s ability to learn task-specific representations of the input data.

Overall, these directions may help improve both the robustness and the task performance of structured input separation approaches for large language models.

6.4 Implications

The results suggest that structural separation between CONTROL and DATA is a promising direction for improving LLM security. However, achieving strong robustness without sacrificing task performance likely requires deeper integration of the DATA channel within the transformer architecture.

Future architectures could explore multi-layer cross-attention mechanisms, trainable data encoders, or hybrid approaches that combine architectural separation with other defense strategies such as instruction hierarchy or prompt filtering.

7 Future Work

Several directions could improve the effectiveness of the SecureLLM architecture.

First, the current implementation performs DATA fusion only once at the embedding level before the transformer layers. Future work could explore deeper integration of the DATA channel through multi-layer cross-attention mechanisms. This would allow the model to reason over DATA information at multiple stages of the transformer, potentially improving task performance while maintaining input separation.

Second, the current design relies on a frozen SBERT encoder to represent the DATA input. While this simplifies training and reduces computational cost, it may limit the model’s ability to capture task-specific information. Future work could investigate partially trainable encoders or joint training strategies to improve the representation of the DATA channel.

Third, the evaluation in this project focuses primarily on sentiment classification. Future experiments should extend the evaluation to additional tasks such as summarization, translation, and entity extraction. This would provide a broader understanding of how architectural input separation affects different types of language tasks.

Finally, SecureLLM could be combined with other prompt injection defenses. For example, architectural separation could be integrated with prompt filtering, instruction hierarchy mechanisms, or retrieval-based safety checks to further reduce the effectiveness of adversarial prompts.

References

- [1] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection, 2023.
- [2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
- [3] Vladislav Lialin, Vijeta Deshpande, Xiaowei Yao, and Anna Rumshisky. Scaling down to scale up: A guide to parameter-efficient fine-tuning, 2024.
- [4] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models, 2022.
- [5] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.
- [6] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models, 2026.